

LeapAI Website — GCP Deployment Guide

Project: LeapAI Website Platform (Next.js + Express + MongoDB + Redis)

Document version: 1.0

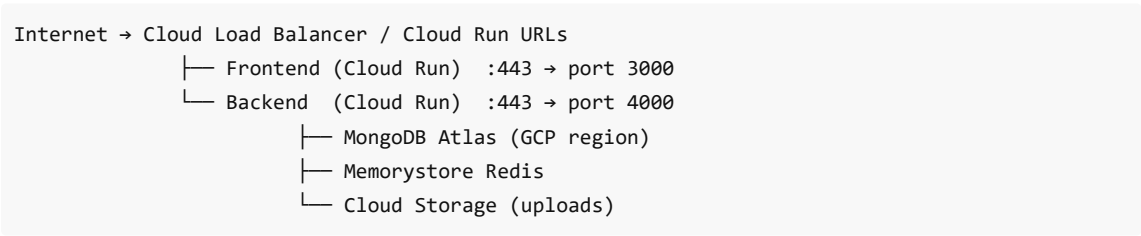
Last updated: July 2026

1. Overview

This guide explains how to deploy the LeapAI website to **Google Cloud Platform (GCP)** using Docker containers.

Component	Technology	Recommended GCP service
Frontend	Next.js 16	Cloud Run
Backend API	Express + Node.js	Cloud Run
Database	MongoDB 7	MongoDB Atlas (on GCP) or Compute Engine VM
Cache	Redis 7	Memorystore for Redis
Uploaded images	Local /uploads	Cloud Storage bucket
Docker images	Custom builds	Artifact Registry
CI/CD (optional)	Cloud Build	Cloud Build triggers

Architecture (recommended):



2. Prerequisites

Before you start, make sure you have:

1. **Google Cloud account** with billing enabled
2. **GCP project** created (example: `leapai-prod`)
3. **gcloud CLI** installed: <https://cloud.google.com/sdk/docs/install>
4. **Docker Desktop** installed locally
5. **Domain name** (optional, e.g. `leapai.ai`)
6. **MongoDB Atlas account** (free tier works for testing): <https://www.mongodb.com/atlas>

Login and set project

```
gcloud auth login
gcloud config set project YOUR_PROJECT_ID
gcloud config set run/region me-central1
```

Tip: Choose a region close to your users. For Saudi Arabia, consider `me-central1` (Doha) or `me-central2` (Dammam) if available.

3. Enable required GCP APIs

Run once per project:

```
gcloud services enable `
  run.googleapis.com `
  artifactregistry.googleapis.com `
  cloudbuild.googleapis.com `
  secretmanager.googleapis.com `
  storage.googleapis.com `
  redis.googleapis.com `
  compute.googleapis.com
```

4. Create Artifact Registry (Docker image storage)

```
gcloud artifacts repositories create leapai-repo `
  --repository-format=docker `
  --location=me-central1 `
  --description="LeapAI website Docker images"

gcloud auth configure-docker me-central1-docker.pkg.dev
```

5. Build and push Docker images

From the project root (`leap website` folder):

5.1 Set variables

```
$PROJECT_ID = "YOUR_PROJECT_ID"
$REGION = "me-central1"
$REPO = "leapai-repo"
$REGISTRY = "$REGION-docker.pkg.dev/$PROJECT_ID/$REPO"
$DOMAIN = "https://leapai.ai"
$API_DOMAIN = "https://api.leapai.ai"
```

5.2 Build backend image

```
docker build -t "$REGISTRY/backend:latest" ./backend
docker push "$REGISTRY/backend:latest"
```

5.3 Build frontend image

Replace URLs with your production domains:

```
docker build `
  --build-arg NEXT_PUBLIC_API_URL=$API_DOMAIN `
  --build-arg NEXT_PUBLIC_SITE_URL=$DOMAIN `
  -t "$REGISTRY/frontend:latest" `
  ./frontend

docker push "$REGISTRY/frontend:latest"
```

6. Set up MongoDB

Option A — MongoDB Atlas (recommended)

1. Create a cluster in **MongoDB Atlas** (choose GCP provider + same region).
2. Create a database user (e.g. `leap`) with a strong password.
3. Allow network access from Cloud Run (use **0.0.0.0/0** for testing, or VPC peering for production).
4. Copy the connection string:

```
mongodb+srv://leap:YOUR_PASSWORD@cluster0.xxxxx.mongodb.net/leapai?retryWrites=true&w=majority
```

Option B — MongoDB on a Compute Engine VM

Run MongoDB in Docker on a VM with a persistent disk. Use only if you need full self-hosting on GCP.

7. Set up Redis (Memorystore)

```
gcloud redis instances create leapai-redis `
  --size=1 `
  --region=me-central1 `
  --redis-version=redis_7_0 `
  --tier=basic
```

Get the Redis host:

```
gcloud redis instances describe leapai-redis --region=me-central1 --format="value(host)"
```

Connection URL format:

```
redis://REDIS_HOST:6379
```

Note: Memorystore is VPC-only. Cloud Run must use a **VPC connector** to reach it.

8. Create Cloud Storage bucket for uploads

```
gsutil mb -l me-central1 gs://YOUR_PROJECT_ID-leapai-uploads
gsutil iam ch allUsers:objectViewer gs://YOUR_PROJECT_ID-leapai-uploads
```

For production, serve uploads through the backend or signed URLs — do not make the bucket fully public unless intended.

9. Store secrets in Secret Manager

Never commit production passwords to Git.

```
# JWT secret (generate a long random string)
echo -n "YOUR_LONG_RANDOM_JWT_SECRET" | gcloud secrets create jwt-secret --data-file=-

# Admin password
echo -n "YOUR_STRONG_ADMIN_PASSWORD" | gcloud secrets create admin-password --data-file=-

# MongoDB URI
echo -n "mongodb+srv://..." | gcloud secrets create mongodb-uri --data-file=-
```

Grant Cloud Run access to secrets:

```
gcloud secrets add-iam-policy-binding jwt-secret \
  --member="serviceAccount:YOUR_PROJECT_NUMBER-compute@developer.gserviceaccount.com" \
  --role="roles/secretmanager.secretAccessor"
```

Repeat for each secret.

10. Deploy backend to Cloud Run

```
gcloud run deploy leapai-backend \
  --image "$REGISTRY/backend:latest" \
  --region me-central1 \
  --platform managed \
  --allow-unauthenticated \
  --port 4000 \
  --memory 512Mi \
  --set-env-vars
"NODE_ENV=production,PORT=4000,CORS_ORIGIN=$DOMAIN,ENFORCE_PROD_SECRETS=true,SEED_ON_START=false" \
  --set-secrets "JWT_SECRET=jwt-secret:latest,MONGODB_URI=mongodb-
uri:latest,ADMIN_PASSWORD=admin-password:latest" \
  --set-env-vars "ADMIN_EMAIL=admin@leapai.ai,REDIS_URL=redis://REDIS_HOST:6379"
```

After deploy, note the backend URL:

```
https://leapai-backend-xxxxx-me-central1.a.run.app
```

Test health:

```
curl https://YOUR_BACKEND_URL/api/public/health
```

11. Deploy frontend to Cloud Run

```
gcloud run deploy leapai-frontend `
  --image "$REGISTRY/frontend:latest" `
  --region me-central1 `
  --platform managed `
  --allow-unauthenticated `
  --port 3000 `
  --memory 1Gi `
  --set-env-vars
  "NEXT_PUBLIC_API_URL=$API_DOMAIN,NEXT_PUBLIC_SITE_URL=$DOMAIN,API_URL=https://YOUR_BACKEND_URL,
```

Test the site:

```
curl -I https://YOUR_FRONTEND_URL
```

12. Map custom domains

12.1 Frontend (leapai.ai)

```
gcloud run domain-mappings create `
  --service leapai-frontend `
  --domain leapai.ai `
  --region me-central1
```

12.2 Backend API (api.leapai.ai)

```
gcloud run domain-mappings create `
  --service leapai-backend `
  --domain api.leapai.ai `
  --region me-central1
```

Add the DNS records shown by GCP to your domain registrar. SSL certificates are provisioned automatically.

13. Production environment variables

Backend (Cloud Run)

Variable	Example	Required
NODE_ENV	production	Yes
PORT	4000	Yes
MONGODB_URI	Atlas connection string	Yes
REDIS_URL	redis://host:6379	Yes

JWT_SECRET	Long random string	Yes
CORS_ORIGIN	https://leapai.ai	Yes
ADMIN_EMAIL	admin@leapai.ai	Yes
ADMIN_PASSWORD	Strong password	Yes
ENFORCE_PROD_SECRETS	true	Recommended
SEED_ON_START	false	Yes (production)

Frontend (Cloud Run)

Variable	Example	Required
NEXT_PUBLIC_API_URL	https://api.leapai.ai	Yes
NEXT_PUBLIC_SITE_URL	https://leapai.ai	Yes
API_URL	Backend Cloud Run URL (internal)	Yes
INTERNAL_API_URL	Same as API_URL	Yes

Important: Rebuild the frontend Docker image whenever `NEXT_PUBLIC_*` values change — they are baked in at build time.

14. Seed content (first deploy only)

Run once after backend is live:

```
# From your local machine, pointing to production API
cd backend
$env:MONGODB_URI="mongodb+srv://..."
$env:ADMIN_EMAIL="admin@leapai.ai"
$env:ADMIN_PASSWORD="YOUR_STRONG_ADMIN_PASSWORD"
npm run seed
```

Or temporarily set `SEED_ON_START=true` on first deploy, then set it back to `false`.

15. CI/CD with Cloud Build (optional)

Create `cloudbuild.yaml` in the project root:

```
steps:
- name: gcr.io/cloud-builders/docker
  args: ['build', '-t', '$_REGISTRY/backend:$COMMIT_SHA', './backend']
- name: gcr.io/cloud-builders/docker
  args: ['push', '$_REGISTRY/backend:$COMMIT_SHA']
- name: gcr.io/cloud-builders/docker
  args:
  - build
```

```

- --build-arg
- NEXT_PUBLIC_API_URL=$_API_URL
- --build-arg
- NEXT_PUBLIC_SITE_URL=$_SITE_URL
- -t
- $_REGISTRY/frontend:$COMMIT_SHA
- ./frontend
- name: gcr.io/cloud-builders/docker
  args: ['push', '$_REGISTRY/frontend:$COMMIT_SHA']
- name: gcr.io/google.com/cloudsdktool/cloud-sdk
  args:
    - run
    - deploy
    - leapai-backend
    - --image=$_REGISTRY/backend:$COMMIT_SHA
    - --region=$_REGION
    - --platform=managed
substitutions:
  _REGISTRY: me-central1-docker.pkg.dev/YOUR_PROJECT_ID/leapai-repo
  _REGION: me-central1
  _API_URL: https://api.leapai.ai
  _SITE_URL: https://leapai.ai

```

Trigger on Git push:

```

gcloud builds triggers create github `
  --repo-name=leap-website `
  --repo-owner=YOUR_GITHUB_ORG `
  --branch-pattern="^main$" `
  --build-config=cloudbuild.yaml

```

16. Alternative: Single VM with Docker Compose

For simpler setups, deploy everything on one **Compute Engine** VM:

1. Create an e2-standard-2 VM (Ubuntu 22.04) with 50 GB disk.
2. Install Docker and Docker Compose.
3. Copy the project to the VM.
4. Create production `.env` with real secrets.
5. Update `docker-compose.yml` URLs to your domain.
6. Run:

```
docker compose up -d --build
```

7. Point domain A record to the VM IP.
8. Use **Caddy** or **nginx** as reverse proxy with Let's Encrypt SSL.

This is easier to start but harder to scale than Cloud Run.

17. Post-deployment checklist

- ☐ Change default admin password (`admin123` → strong password)
- ☐ Set `ENFORCE_PROD_SECRETS=true` on backend
- ☐ Set `SEED_ON_START=false` after initial seed
- ☐ Verify `https://leapai.ai` loads
- ☐ Verify `https://leapai.ai/dashboard/login` works
- ☐ Test maintenance mode toggle in dashboard
- ☐ Verify `https://leapai.ai/llms.txt` and `/robots.txt`
- ☐ Run E2E tests against production URL
- ☐ Set up Cloud Monitoring alerts
- ☐ Configure automated MongoDB Atlas backups

18. Troubleshooting

Problem	Solution
Dashboard "Failed to fetch"	Check <code>NEXT_PUBLIC_API_URL</code> and backend CORS (<code>CORS_ORIGIN</code>)
Maintenance mode not redirecting	Ensure <code>API_URL</code> points to backend; check backend rate limits
Arabic text shows ????	Re-run <code>npm run sync</code> with UTF-8 encoding; reset site settings
Uploads disappear after restart	Use Cloud Storage or persistent volume for <code>/app/uploads</code>
502 on Cloud Run	Check logs: <code>gcloud run services logs read leapai-backend</code>
Redis connection failed	Attach VPC connector to Cloud Run for Memorystore

View logs

```
gcloud run services logs read leapai-backend --region me-central1
gcloud run services logs read leapai-frontend --region me-central1
```

19. Useful commands summary

```
# Build and push
docker build -t $REGISTRY/backend:latest ./backend && docker push $REGISTRY/backend:latest
docker build --build-arg NEXT_PUBLIC_API_URL=$API_DOMAIN --build-arg
NEXT_PUBLIC_SITE_URL=$DOMAIN -t $REGISTRY/frontend:latest ./frontend && docker push
$REGISTRY/frontend:latest

# Deploy
gcloud run deploy leapai-backend --image $REGISTRY/backend:latest --region me-central1
gcloud run deploy leapai-frontend --image $REGISTRY/frontend:latest --region me-central1

# List services
```



```
gcloud run services list --region me-central1

# Local test before upload
docker compose up --build -d
powershell -ExecutionPolicy Bypass -File scripts/e2e-test.ps1
```

20. Support contacts

- **Admin dashboard:** /dashboard/login
 - **Default admin (change in production):** admin@leapai.ai
 - **Project repo:** leap website monorepo
 - **GCP docs:** <https://cloud.google.com/run/docs>
-

End of document